HW-3

Web Search Engines

Utkarsh Jain
[redacted]
20 November 2023

Table of Contents

Table of Contents 2

1. Background 3

2. Inverted Index Creation 4

2.1 Intermediate Files 4

2.2 Cleaning Intermediate Files 4

2.3 Sorting Intermediate Files 5

2.4 Merging Sorted Files 5

2.5 Compression 6

2.5.1 Dynamic Block Sizes 6

2.6 RAM Usage 7

3. Query Processing 8

3.1 Low-Level Document Manager 8

3.1.1 Inverted Lists by Word 8

3.1.2 Term Frequency in a Word 8

3.1.3 In-memory Data Structures 8

3.1.4 Fetching Documents by Doc ID 9

3.1.5 Caching 9

3.2 Query Processor 10

3.2.1 Conjunctive Query 10

3.2.2 Disjunctive Query 10

3.2.3 Snippet Generation 11

3.3 Other Good Code Writing Practices and Special Mentions 11

3.4 Alternate Implementation and Future Improvements 15

4. Related Statistics 17

5. Acknowledgements 18

1. Background

An inverted index is a foundational data structure in information retrieval systems like web search engines. It aims to map content, like words, to their locations in documents. It's called "inverted" because it flips a traditional index, which lists documents and their contents, allowing users to search by content to find documents.

The necessity of an inverted index stems from the need for efficiency in searching vast amounts of data. With the exponential growth of the internet, manually searching through documents for specific terms could be more practical. The inverted index automates this by cataloguing each word and listing all the records in which that word appears. When a query is made, the search engine uses the index to quickly retrieve a list of documents containing the search terms, often in milliseconds.

Conjunctive queries (AND) and disjunctive queries (OR) are two search operations that benefit from an inverted index. Conjunctive queries require that all terms be present in the resulting documents, while disjunctive queries return documents containing any terms. These operations are akin to set intersections and unions, respectively, and are efficiently performed thanks to the structured nature of inverted indexes.

For instance, when a user inputs "chocolate AND cookie," they make a conjunctive query; they want results where both terms are present. In contrast, "chocolate OR cookie" is a disjunctive query; the user is looking for documents containing either term. The inverted index allows the search engine to find the list of documents for each search term quickly and then combine those lists as needed to satisfy the query.

This system provides rapid access to information and supports a wide range of query complexities, from single keyword searches to intricate multi-term queries. It is essential for delivering relevant search results and a seamless user experience.

In conclusion, the inverted index is indispensable for the speed and scalability of search engines. Its ability to facilitate conjunctive and disjunctive queries ensures users can conduct searches that align with their specific informational needs. With such a system, search engines could provide the fast, relevant, and comprehensive results users expect in the digital age.

2. Inverted Index Creation

Assignment 2 dealt with creating and compressing the Inverted Index. The approach followed in this report creates intermediate files combining 1000 documents. This intermediate index is then cleaned since it contains polluted terms. This process ends by sorting these cleaned indexes. These are merged using a custom-written IO Efficient merge routine and then compressed.

2.1 Intermediate Files

This function processes a TREC format document and creates intermediate index files for documents and their content. The goal is to split the content into smaller, manageable chunks (in this case, 1000 documents each) and store the indexed information. Intermediate indexes are created to manage memory usage and periodically write the data to disk.

This function is careful with resource management, using memory checks to avoid overconsumption and writing data to disk in chunks. It also uses data structures (unordered_map and unordered_set) to manage the index and keep track of unique words and document details efficiently.

The function proceeds by initialising variables and opening required files. It proceeds to read the `.trec` file; in doing so, it uses tags such as `<DOC>`, `</TEXT>`, `<DOCNO>` to keep track of where the reader is.

As expected, apart from completing 1000 documents, the function dumps everything when the memory threshold is reached. While creating intermediate files, this function also makes the DocToUrl table. It keeps writing document sizes, where the document starts in the big `.trec` file (required later for snippet generation) and document URLs.

2.2 Cleaning Intermediate Files

This function holds the responsibility of cleaning the intermediate postings. There can be multiple words like ‘word7anotherword’; this function will break this into ‘word’ and ‘anotherword’. When it does, it uses the same inverted list for all the split words since they occurred in the same document with just as much frequency.

With a similar design to the previous function, it initialises required variables and opens necessary files. It then keeps opening individual intermediate postings, parsing them line by line and cleaning the words as

needed. It keeps reading the folder with intermediate postings until there are no more intermediate files to read. It also uses data structures (`unordered_map` and `unordered_set`) to manage the index and keep track of unique words and document details efficiently.

As expected, if and when RAM usage goes over the threshold, the loop is halted mid-way, all the buffers are emptied and then, using the `goto` statement executions, returns right where it left off.

2.3 Sorting Intermediate Files

This function takes unsorted intermediate index files, sorts them, and writes them out. The function does not explicitly sort the postings and terms; instead, it puts them in a map, which implements a Red-Black Tree internally. These are inherently sorted trees while also maintaining uniqueness.

With no surprises in its design, it starts by initialising variables and opening the required files. It reads the cleaned files line by line and puts the words and their indexes into the map. This map is later emptied by writing its content to a new file. If we empty the map, we lose the ordering created so far, and the new reads of this posting will not be sorted. This problem is solved by writing a new file in case of an intermediate break. Because no matter the files, they will be merged later.

Like all others, it has a RAM threshold and handles it accordingly.

2.4 Merging Sorted Files

As the name suggests, this function merges the sorted and cleaned intermediate files. It iteratively merges sorted index files into a single, comprehensive index file.

What this function attempts to do differently is it keeps on creating new files but also removes older files and folders generated between merges. Moreover, when dealing with an odd number of files, it moves the last file into the new directory and names it accordingly so it can be merged later.

In this function, if and when the RAM usage hits the threshold, it's appended to the previously opened file rather than creating a new one. Otherwise, if we keep creating new files, the code might keep on finding files to merge and never converge to a single file. After writing it, it resumes right where it left off using `goto`. A problem worth mentioning is that when using comparator operators such as '<' and '>', there was a problem with the ordering, leading to duplicates and using the string.compare function led to

a file with no duplicates and a proper order. This suspicious behaviour might be due to the implementation of Apple's Clang compiler.

2.5 Compression

This function compresses the inverted index and creates a lexicon table, a crucial step in optimising the search engine for efficient storage and retrieval.

The function starts initialising variables and opening necessary files. It also loads the docTable created during the generation of intermediate postings into a vector. The function reads this massive index, line by line, removing terms and combining DocIds & Frequencies into fixed-sized blocks. These combined blocks are preceded by metadata containing [total blocks, block size, last block size]. Final compressed list looks like [{MetaData}{DocIDs}{Frequencies}{DocIds}{Frequencies}...]. It might appear useless to store block size, but it was later used to analyse if there was a need for dynamic block size. These combined vectors are then compressed using var-byte compression.

After encoding for an inverted list, the maxxscore for each term is also calculated using the BM25 method to be used later to speed up disjunctive queries. This computation of maxxscore for each term puts a significant penalty on the performance of compression since loading the entire doc table has spiked the RAM usage, so there are more frequent disk writes to reduce RAM Usage, but even after writing out buffers, the memory usage is still higher since the table is still in memory. To speed up, while searching for the document that gives Maxxscore, complete BM25 calculations are not performed. For a given term, the Maxxscore document can be defined as:

$$\underset{doc}{\text{argmax}} = \left(\frac{\text{freq[doc]}}{\text{freq[doc]} + len(\text{doc})} \right)$$

We find the document that yields the max score using this simplified computation and then perform a full BM25 computation on it. This simplification works out for us since the term remains constant.

2.5.1 Dynamic Block Sizes

Dynamic Block sizes were considered using the following analysis.

Let the number of seeks in the worst case with a fixed block size be defined as

$$seeks(Block\ Size) = \frac{len(Inverted\ List)}{Block\ Size} + Block\ Size$$

The first term is total blocks, and then by reading the block ID by id, we can have at most Block Size reads. Now, we want to minimise the number of seeks, so we take the derivative of this equation.

$$\frac{d}{d(Block\ Size)}\left(Block\ Size + \frac{len(Inverted\ List)}{Block\ Size}\right) = 1 - \frac{len(Inverted\ List)}{Block\ Size^2}$$

This tells us that this equation minimises at

$$Block\ Size = \sqrt{len(Inverted\ List)}$$

Now, when we compare the new seek equation with the older one.

$$2 \cdot \sqrt{len(Inverted\ List)} \leq \frac{len(Inverted\ List)}{Block\ Size} + Block\ Size$$

For any value of Block Size, we have fewer seeks than a fixed block size inverted lists when trying to seek while hoping. Hence, this dynamic Block Size was used in this assignment. An interactive version demonstrating how the value of block sizes, number of blocks and number of worst-case seeks needed on an inverted list vary can be found here.

2.6 RAM Usage

RAM Usage has been calculated for the current process in Megabytes and can be limited as such. For the purpose of this assignment, a platform-dependent library `<mach>` has been used. However, checking RAM usage at every iteration would give a significant performance hit, so RAM checks are made at every 512 iterations; this number is adjustable by modifying the `MEM_CHECK_CNT` variable.

3. Query Processing

3.1 Low-Level Document Manager

This class handled opening/closing, reading and processing the three indexes we made in Assignment 2.

3.1.1 Inverted Lists by Word

Two Implementations were written for this function. The original function would read the entire list in one go. However, the assignment wants the decompression done block by block instead of decompressing the entire list. The required implementation takes in the word and checks if the entire list or its fragment exists in the cache. If it exists in the cache, it starts feeding from it. However, if only a portion of the inverted list exists in the cache, then the function will only fetch the remaining blocks if and when required and update the cache accordingly. In the original implementation, entirely read lists would be cached as well.

We must maintain a state of till where we have read it so far. However, this problem is solved by using a coroutine.

3.1.2 Term Frequency in a Word

Similarly, there are two implementations where both look up in the cache for the inverted list. They perform hops across the blocks, and when the beginning of a block is greater than the current docID, the previous block is iterated to find the ID. In both implementations, this function returns the frequency of the term in the document. If the term does not exist, it produces a null value. The only slight difference is that the block-by-block approach would first try to use the blocks decompressed in the cache and then fetch new blocks if required. The jumps over the intermediate frequency blocks are hidden from the user. Whenever a block is suspected to have a frequency, it's binary searched to find the target ID.

3.1.3 In-memory Data Structures

As recommended, the Document Table and Lexicon Table are kept in memory. Design consideration was made considering the space/speed trade-off. In this assignment, a vector is used instead of an unordered_map to save

memory, and since both the Document Table and Lexicon Table are sorted, we can use Binary search to find documents.

$$\log_2(3{,}213{,}835) \approx 21.61$$

In the worst case, we need to make 22 checks. However, this is only required for the Lexicon Table. Since the Document Table is sorted by ID, we can directly have the vector return the ID—no need to perform a binary search. Hence, we have O(log2(n)) for the lexicon table and O(1) for the Document Table.

3.1.4 Fetching Documents by Doc ID

To generate snippets, we need the document text. The position at which the doc text starts is also stored during the page table creation. During snippet generation, when the function gets a docId, it fetches the start position from the page table and jumps to the position in the .trec file. It gets the entire document text from the file.

3.1.5 Caching

There are two cache implementations; one is a trivial simple cache that fetches the entire list by term. Second, the non-trivial cache stores a fragmented inverted list. It stores extra information about the inverted list, such as how much has been read from the file and how many blocks have been read so far. Functions in the implementation use this information to fetch it only after the entire cached list has been read and update it accordingly. Whenever a cached list is modified, it's done so by adding a block or removing the entire list.

A simple cache was also built to cache document texts to reduce fetches to the big trec file.

3.2 Query Processor

Query Processor runs conjunctive and disjunctive queries defined in the assignment.

3.2.1 Conjunctive Query

Conjunctive Query also has two implementations. One that uses the entire inverted list in one go. It was supposed to be performance optimised using the new c++20 added std::reduce defined in the <numeric> to use parallel processing to intersect all inverted lists. The code is still left there as a comment

In the second implementation, based on block-by-block decompression, each inverted index is only read through once. The hopping technique mentioned in class is used. With the added functionality of co-routines, it works great!

After which, it sends the intersected IDs to another function to calculate BM25 scores. While calculating BM25 scores, all calculated scores are added to a min-heap priority queue. As soon as the priority queue has more than K values, the queue pops elements to bring down the queue size to K. It keeps the memory usage low and gets us the top K results by just popping elements from the priority queue and reversing it.

These top K doc IDs are then sent for snippet generation.

3.2.2 Disjunctive Query

The disjunctive query's two implementations only differ in the MaxxScore version. In the non-block-by-block decompression method, the first algorithm tries to load all docIds for all terms. Sort them based on their max BM25 scores. These sorted DocIds and words are given to the MaxxScore algorithm. Even when doing block-by-block decompression, the words are sorted ingeniously based on their BM25 scores, and only these words are given to MaxxScore.

After doing MaxxScore, we get top-K docs, which are sent for snippet generation.

3.2.2.1 The MaxxScore Algorithm

The General Algorithm used in both iterations of MaxxScore is described here. In order of the highest Maximum BM25 scores, the algorithm reads docId individually. If it has encountered this docId before, the algorithm skips

it*. If the docID has not been encountered before, It calculates BM25 scores for this doc ID using all the words in the query. If the term is not in the document, the algorithm assigns it a BM25 score of 0, so that doesn't affect the accumulation of scores. A min-heap priority queue keeps top K document IDs, similar to the conjunctive query. We only consider pushing a document to the priority queue if the priority queue has space or the intermediate score of this Doc ID is greater than the minimum value in the priority queue. Whenever a new value is added to this priority queue, we check to see if we can add a term to the non-essential collection in case the threshold has gone up. If the current term becomes non-essential, it's skipped immediately.

*We can make this optimisation since we find BM25 scores of the docId as if it did have those words, which also includes terms with a low max BM25 score. In a way, this is the true BM25 score of a doc ID with respect to the query.

3.2.3 Snippet Generation

The algorithm tries to find the first K sentences from the document with the query words.

In this text, it now looks for the word in a query; if it finds the word, it attempts to find the sentence in which it occurs. As and when the sentence is found, the start and stop positions of this sentence are stored. When we look for the next word in the query, we first check if the number of individual sentences found is greater than K. Then, we only accept this word if its occurrence position will not lead to a new sentence that occurs further than the previous K sentences. Another check is made; if it occurs in one of the previous sentence lengths, it is unnecessary to find its sentence. If this causes us to load a new sentence into the store, we remove the sentence that occurs furthest away from the start position. If there are more than required sentence entries, the algorithm tries to prioritise the sentences with more query words.

3.3 Other Good Code Writing Practices and Special Mentions

The implementation uses classes and does not create unnecessary public variables or functions. The `const` keyword has been used as and when required to prevent unwarranted data modification. Variables and Functions

are aptly named. Unnecessary implementation details are encapsulated accordingly.

This part will also talk about offloading the task of state management to the C++ compiler rather than implementing state management manually. It uses C++ coroutines.

After using a C++ routine, the following function implementation is possible.

```
IntGenerator getVal() {
    size_t start = InvertedListBegin(word);
    size_t stop = InvertedListEnd(word);
    int val = 0, shift = 0;
    for (size_t i = start; i < stop; i++) {
        const short b = getByte(i);
        if (b < 128) {
            val = val + (b << shift);
            shift = shift + 7;
        } else {
            val = val + ((b - 128) << shift);
            co_yield val;
            val = 0; shift = 0;
        }
    }
    co_yield -1;
}
```

This can be used as follows

```
int main() {
    auto gen = getVal();
    for (int i = 0; i < 10; ++i)
        if (gen.next())
            std::cout << "Posting val: " << gen.getValue() << std::endl;
    return 0
}
```

What makes this great is that getValue() will continue processing from right where it leaves off. This removes the load of maintaining the state by the student. Any function that returns an int value can be used with the return type Intgenerator to get a similar functionality. The code for Intgenerator is below.

```
struct IntGenerator {
    struct promise_type {
        int current_value = 0;

        std::suspend_always yield_value(int value) {
            current_value = value;
            return {};
        }
        std::suspend_always initial_suspend() { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        IntGenerator get_return_object() { return IntGenerator{this}; }
        void return_void() {}
```

```
        void unhandled_exception ) { std::exit(1); }
    };

    IntGenerator(promise_type* p) :
handle(std::coroutine_handle<promise_type>::from_promise(*p)) {}
    ~IntGenerator() { if (handle) handle.destroy(); }
    IntGenerator(const IntGenerator&) = delete;
    IntGenerator& operator=(const IntGenerator&) = delete;
    IntGenerator(IntGenerator&& other) : handle(other.handle) { other.handle = nullptr; }
    IntGenerator& operator=(IntGenerator&& other) {
        if (this != &other) {
            if (handle) handle.destroy();
            handle = other.handle;
            other.handle = nullptr;
        }
        return *this;
    }

    std::coroutine_handle<promise_type> handle;

    bool next() {
        if (!handle.done()) {
            handle.resume();
            return true;
        }
        return false;
    }

    int getValue() const {
        return handle.promise().current_value;
    }
};
```

This boilerplate code can be used for any functions returning ints.

Coroutines are a general control structure where flow control is cooperatively passed between two different routines without returning. In the context of the generator struct, it allows us to write a function that can produce a sequence of values.

When we call a function that contains a co_yield statement, it returns a generator object. You can then call move_next() on this object to resume the coroutine and produce the next value in the sequence. The coroutine runs until it hits the co_yield statement, saving its state and suspending it. The yielded value can be retrieved with current_value().

This is powerful because it allows writing code that generates complex sequences of values in an easy-to-understand and efficient way. The coroutine only runs when asked for the next value and only uses memory to store its current state. This makes it an excellent tool for writing code that

deals with sequences of values, like iterators, streams, or asynchronous tasks.

In the bigger picture, coroutines are a powerful tool for managing concurrency in a program. They allow writing asynchronous code in a synchronous style, which can make code easier to understand and reason about. They're instrumental in scenarios where multiple tasks that can run independently but need to coordinate in some way, such as IO-bound tasks, user interfaces, or simulations, need to be managed. In these cases, coroutines can help you write efficient, scalable, and maintainable code.

The generator is explained below.

`generator<T>`: This template struct can generate values of type T. In our case, T is int.

`promise_type`: This is a nested struct within the generator that represents the coroutine's state. It includes the current value being generated (T value) and several methods that define the behaviour of the coroutine.

handle_type: This is a type alias for `std::experimental::coroutine_handle<promise_type>`, which is a handle to the coroutine. It's used to control the execution of the coroutine.

`get_return_object()`: This method is called when the coroutine is first invoked and returns the generator object that the coroutine will yield values through.

`initial_suspend()`: This method is called at the start of the coroutine, and it returns an object with a await_ready, await_suspend and await_resume method. In this case, `std::experimental::suspend_always` is returned, which means the coroutine is initially suspended and won't run until it is explicitly resumed.

`final_suspend()`: This method is called after the coroutine is done. It also returns `std::experimental::suspend_always`, which means the coroutine is suspended after it finishes. This allows us to check whether the coroutine is done before resuming it again.

`unhandled_exception()`: This method is called when an exception is thrown inside the coroutine. In this case, it calls `std::terminate()`, which terminates the program. This can be replaced with more robust error handling.

`return_void()`: This method is called when the coroutine reaches a `co_return;` statement. In this case, it does nothing because the coroutine is a generator that yields values with `co_yield` instead of returning a value with `co_return`.

`yield_value(T value)`: This method is called when the coroutine reaches a `co_yield` value; statement. It stores the yielded value and suspends the coroutine.

`move_next()`: This method resumes the coroutine and returns whether the coroutine has more values to yield.

`current_value()`: This method returns the current value yielded by the coroutine.

`generator(generator const&) = delete;`: This line deletes the copy constructor of generator. This is because copying a generator doesn't make sense in this context, as it would involve copying the coroutine's state.

`generator(generator && rhs) : coro(rhs.coro) { rhs.coro = nullptr; }`: This is the move constructor of generator. It takes over the ownership of the coroutine handle from another generator.

`~generator() { if (coro) coro.destroy(); }`: This is the destructor of the generator. It destroys the coroutine when the generator is destroyed.

This project uses coroutines whenever docIds or frequencies need to be read sequentially. Coroutines allow for amortised O(1) sequential reads. It's amortised since it's reading from cache most of the time and when cache is exhausted it either tries to add a block or ends the generator. If these coroutines use cache, cache synchronisation needs to be added, so that whenever a coroutine resumes it has an upto date cache. This will prevent unnecessary fetches, since one coroutine can fetch a fetch a block of entries and the other coroutines will behave as if the block was always there.

3.4 Alternate Implementation and Future Improvements

One of this assignment's most frustrating parts is maintaining state during block-by-block decompression. There are many ways to do so; one is using the cache to maintain state, and another could be to wrap docIds in a class wrapper that maintains the state for each docID so every time a function next() is run, it can fetch the next docId by decompression.

For future development, I would like to use bit-aligned compression so that there can be block jumps. With VarByte it's impossible to know when the block ends naturally. One has to traverse and keep a count of bytes > 128 manually. As and when required, multiple lower-level functions can also be rewritten and rethought to use coroutines. Between various implementations, to get the best efficiency, it's best to re-think procedures to avoid redundant code.

4. Related Statistics

It takes about 2 hours to generate intermediate indexes. It takes less than an hour to clean them. It takes less than an hour to sort them. It takes about an hour to merge them and 40-ish minutes to compress them. These timings are found when I push the code to write outlines as soon as it computes them. When run concurrently, it takes about 1 hour 40 minutes. With varying RAM, these times can be reduced further. I felt it's worthwhile to mention that during the lowest memory usage run, merging only took about 20 MB of RAM at maximum. Moreover, the checks for RAM Usage make the code slower and maybe a counter can be put to regulate how often these RAM checks are made. Lexicon Table gets created at compression time. It stores where the inverted list for a word starts and uses Tellp to store exact binary offsets to where the list ends and how many times the word occurs in the index. The docId to URL table gets created when intermediate postings are generated.

5. Acknowledgements

I want to acknowledge that StackOverflow was taken as a reference point.